UNIVERSIDAD
NACIONAL
DEL NORDESTE

Facultad de Ciencias Exactas, Naturales y Agrimensura

Carrera: Licenciatura en Sistemas de Información

Asignatura: Ingeniería de Software II

Integrantes
Sosa Diana Abril
Valdes Nicolas Eduardo

Índice

Índice **2**
Índice de tablas **3**
Índice de imágenes **3**
1. INICIO **4**
1.1 Introducción **4**
1.2 Objetivo **4**
1.3 Alcance **4**
1.3.1 Funcionalidad I: Gestión Académica e Interactividad 4
1.3.2 Funcionalidad II: Automatización y Validación de Intercambios 4
2. Metodología **5**
2.1 Ciclo de vida del proyecto **5**
2.1.1 Product Backlog 5
2.2 Evaluación de Arquitecturas. **6**
2.2.1 Multi-Tier (3 niveles) 6
2.2.1.1 Ventajas 7
2.2.1.2 Desventajas 7
2.2.1.3 Conclusión 7
2.3 Especificación de Requisitos de Software (ERS) **8**
2.3.1 Perspectiva del Producto 8
2.3.2 Funciones del Producto 8
2.3.3 Características de los Usuarios 8
2.3.4 Restricciones 9
2.3.5 Requerimientos del Sistema 9
2.3.5.1 Requisitos Funcionales (RF) 9
2.3.5.2 Requisitos No Funcionales (RNF) 10
2.4 Plan de Riesgos **11**
2.5 Cronograma **14**
2.5.2 Sprint 1: Cimentación y Análisis (Semanas 1-2) 14
2.5.3 Sprint 2: Diseño detallado y funcionalidad básica (Semanas 3-4) 14
2.5.4 Sprint 3: Desarrollo Principal - Parte A (Semanas 5-6) 15
2.5.5 Sprint 4: Desarrollo Principal - Parte B (Semanas 7-8) 15
2.5.6 Sprint 5: Pruebas, Trazabilidad y Cierre (Semanas 9-10) 15
2.6 Organización de Sprints en ClickUp **16**
2.7 Modelo de datos **17**
3. Herramientas utilizadas **17**
3.1 Stack Tecnológico **17**
3.1.1 Frontend: Next.js 15 17
3.1.2 Backend: NestJS + TypeScript 18
3.1.3 Base de Datos: PostgreSQL (Supabase) 18
3.1.4 ORM: Prisma 18

3.1.5 Generación de PDF: Puppeteer 18
3.1.6 Notificaciones: Supabase Realtime + Nodemailer 18
3.1.7 Testing (Pruebas de Calidad): Jest + Playwright 19
3.1.8 Animaciones y Diseño: Framer Motion, Stitch y UX-UI-pro 19
3.1.9 Gestión de Proyecto: ClickUp 19
3.1.10 Comunicación del Equipo: Discord 19
3.1.11 Diseño y Arquitectura: LucidSpark 19
4. Resultados 20
4.1 Diagrama de Casos de uso 20
4.2 Conversaciones 20
4.2.1 Conversación 1: Incorporar alumno a comisión 21
4.2.2 Conversación 2: dar de baja un alumno 21
4.2.3 Conversación 3: Asignar horario a comisión 22
4.3 Diagramas de Secuencia 23
4.4 Contrato de Operaciones 24
4.4.1 CO-01: Gestión de Alumnos — Incorporar Alumno 24
4.4.2 CO-02: Asignación de Horarios y Eventos al Calendario 25
5. Anexos 25
6. Bibliografía 26

Índice de tablas

Tabla 1 Product Backlog del sistema de intercambio de comisiones
5
Tabla 2. Plan de Riesgos.
11
Tabla 3 Conversación 1: Incorporar alumno a comisión 21
Tabla 4 Conversación 2: Dar de baja un alumno 21
Tabla 5 Conversación 3: Asignar horario a comisión 22
Tabla 6 Contrato de Operación CO-01: Incorporar Alumno 24
Tabla 7 Contrato de Operación CO-02: Asignación de Horarios y Eventos 25

Índice de imágenes

Ilustración 1 Diagrama de arquitectura Multitier 7
Ilustración 2 Organización de sprints en ClickUp 16
Ilustración 3 Modelo de datos 17
Ilustración 4 Diagrama de casos de uso 20
Ilustración 4.1 Diagrama de secuencia: Incorporar alumno a comisión 24
Ilustración 4.2 Diagrama de secuencia: Dar de baja a alumno de comisión 24
Ilustración 4.3 Diagrama de secuencia: Asignar horario a comisión 25

1. INICIO

1.1 Introducción

El presente trabajo de campo se enmarca en el desarrollo de un **Sistema de Matching de Comisiones**, una plataforma web diseñada para optimizar y agilizar el intercambio de vacantes entre estudiantes académicos. El objetivo principal del proyecto es mitigar las dificultades logísticas que enfrentan los alumnos al intentar coordinar sus horarios de cursada, brindando una solución automatizada, transparente y eficiente.

1.2 Objetivo

El objetivo del proyecto es desarrollar una plataforma que tenga un sistema intuitivo que optimice la gestión del intercambio de comisiones para los estudiantes de la universidad que tengan ese deseo y no cuenten con algún método formal que sirva de intermediario para poder hacerlo. Para lograrlo, el sistema permite que el estudiante cargue sus comisiones actuales y visualice su agenda mediante un calendario personal en donde aparecerán los horarios correspondientes a su comisión. Además, la plataforma contará con un sistema de notificaciones en tiempo real por mail que alerta sobre propuestas y confirmaciones, lo que asegura la comunicación fluida entre las partes. Por último, el sistema garantiza que el proceso de intercambio y confirmación sea seguro mediante la generación de comprobantes en formato PDF, brindando una trazabilidad de la operación realizada.

1.3 Alcance

1.3.1 Funcionalidad I: Gestión Académica e Interactividad

Esta funcionalidad se centra en la administración del curso y la comunicación activa entre los integrantes de la comisión.

- **Gestión de Alumnos:** Provee al docente las herramientas para el control del padrón (incorporación, consulta y baja de estudiantes) dentro de su comisión específica.
- **Asignación de Cronograma:** El profesor tiene la facultad de cargar horarios de clases y fechas de exámenes parciales.
- **Mensajería Interna:** Implementación de un canal de chat directo y seguro dentro de la plataforma, permitiendo la comunicación alumno-alumno y alumno-profesor.
- **Visualización de Perfil y Agenda:** * **Agenda Personal:** Calendario interactivo donde el usuario visualiza de forma automática sus eventos académicos cargados por el docente.

1.3.2 Funcionalidad II: Automatización y Validación de Intercambios

Este módulo gestiona la parte lógica del "matching" y la salida formal de datos.

- **Centro de Notificaciones:** Sistema de alertas vía correo electrónico que informa al usuario en tiempo real sobre nuevas propuestas de intercambio, estados de solicitud y confirmaciones de match.

- **Generación de Comprobantes:** Motor de salida documental que crea automáticamente archivos en formato PDF. Estos documentos contienen los datos de los estudiantes y las comisiones permutadas, asegurando la trazabilidad y validez del proceso.

Herramienta de Gestión: Tablero de Tareas

Para garantizar el orden y seguimiento del flujo de trabajo, el sistema incluye:

- **Tablero de Control Estilo Kanban:** Una interfaz visual (basada en columnas de estados como "Pendiente", "En Proceso" y "Finalizado") donde el usuario puede llevar el control de sus tareas personales y académicas, permitiendo una organización ágil de sus actividades diarias.

2. Metodología

2.1 Ciclo de vida del proyecto

Se selecciona el marco de trabajo Scrum debido a su enfoque que nos permite integrar y validar de manera constante la lógica de la agenda y las comisiones. Este ciclo facilita la entrega de las funcionalidades asegurando la trazabilidad entre diseño y código. Además, su estructura mediante el uso de sprints permite incorporar cambios y ajustes necesarios a partir del feedback recibido en cada etapa, garantizando que el producto final se adapte a las necesidades que el cliente quiere solucionar a partir del sistema.

2.1.1 Product Backlog

Tabla 1. Product Backlog del sistema de intercambio de comisiones. Listado priorizado de historias de usuario que define el alcance y la planificación del sistema.

N° HU	Historia de Usuario	Descripción	Prioridad	Puntos
HU-1	Visualización de comisiones disponibles del estudiante	Como estudiante quiero visualizar las comisiones a las que fui asignado para ver el detalle de cada una.	Alta	3
HU-2	Generación de comprobante PDF	Como estudiante, quiero tener un comprobante del intercambio para tener constancia del cambio realizado.	Media	7
HU-3	Notificación de match por correo	Como estudiante quiero recibir una notificación en mi correo cuando se confirme un intercambio para estar informado.	Media	7
HU-4	Visualización de agenda y calendario	Como estudiante quiero ver mis horarios y eventos académicos en un calendario personal.	Media	5
HU-5	Visualización de perfil	Como estudiante quiero visualizar mis datos personales y académicos para tener un resumen de mi información en el sistema.	Alta	3
HU-6	Carga de materias con correspondiente comisión	Como estudiante quiero ver el listado de mis materias con sus comisiones asignadas para conocer mis horarios, aula y profesor.	Alta	5
HU-7	Centro de notificaciones	Como estudiante quiero recibir notificaciones dentro de la plataforma sobre propuestas de intercambio y confirmaciones de match para estar al tanto de novedades.	Media	6
HU-8	Gestión de alumnos — incorporar alumno	Como profesor quiero incorporar alumnos a mi comisión para registrarlos en el sistema.	Alta	5
HU-9	Gestión de alumnos — consultar alumno	Como profesor quiero consultar los alumnos de mi comisión buscando por su numero de usuario o nombre para ver su información.	Alta	3
HU-10	Gestión de alumnos — dar de baja alumno	Como profesor quiero dar de baja a un alumno de mi comisión para mantener actualizado el listado.	Alta	4
HU-11	Asignación de horarios y eventos al calendario	Como profesor quiero cargar horarios de clase, parciales y entregas de TPs para que mis alumnos los visualicen automáticamente en su calendario.	Alta	6
HU-12	Mensajería interna	Como usuario quiero enviar y recibir mensajes directos dentro de la plataforma con un alumno o con un profesor para comunicarme sin salir del sistema.	Baja	8

2.2 Evaluación de Arquitecturas.

2.2.1 Multi-Tier (3 niveles)

La arquitectura multi-tier organiza el sistema en capas físicamente separadas, donde cada nivel se despliega de forma independiente y se comunica con los demás a través de la red.

Nivel de presentación: Next.js actúa como cliente, consumiendo los endpoints expuestos por el backend mediante HTTP. Corre de forma completamente independiente y puede desplegarse en plataformas como Vercel sin ninguna dependencia del servidor de aplicaciones.

Nivel de aplicación: NestJS concentra toda la lógica de negocio y expone una API REST. Es el único componente que tiene acceso directo a la base de datos, actuando como intermediario obligatorio entre la interfaz y los datos. Esto garantiza que el frontend nunca acceda directamente a la base de datos, lo cual es una decisión de seguridad deliberada.

Nivel de datos: Supabase/PostgreSQL opera como servicio gestionado en la nube. Este punto es clave: Supabase por su propia naturaleza tecnológica reside en servidores externos, lo que hace que la separación en tiers no sea solo una decisión de diseño sino una consecuencia directa del stack elegido. No existe la opción de colapsar este tier con los demás.

Esta distinción es importante: no se trata únicamente de separar el código en carpetas o capas lógicas, sino de que cada tier vive en su propio proceso, en su propio servidor, y si uno falla los demás continúan funcionando de forma independiente.

2.2.1.1 Ventajas

- Separación clara de responsabilidades por nivel.
- Seguridad reforzada: el frontend nunca toca la base de datos directamente.
- Despliegue independiente: el equipo puede actualizar el backend sin afectar la interfaz y viceversa.
- Escalabilidad suficiente para el alcance funcional definido.
- Complejidad operativa controlada, adecuada para el tamaño del equipo.

2.2.1.2 Desventajas

- Escalado menos granular que microservicios.
- El backend central puede concentrar más responsabilidades si no se mantiene orden modular interno.

2.2.1.3 Conclusión

Es la opción más adecuada para el proyecto porque el propio stack tecnológico seleccionado impone naturalmente esta separación: Supabase vive en la nube por definición, NestJS expone una API que Next.js consume por red. La arquitectura multi-tier no fue una decisión

arbitraria sino la consecuencia lógica de las tecnologías elegidas, a las que además se suman beneficios concretos como la entrega incremental, la claridad estructural para el mantenimiento y la posibilidad de evolución futura sin rehacer la base del sistema

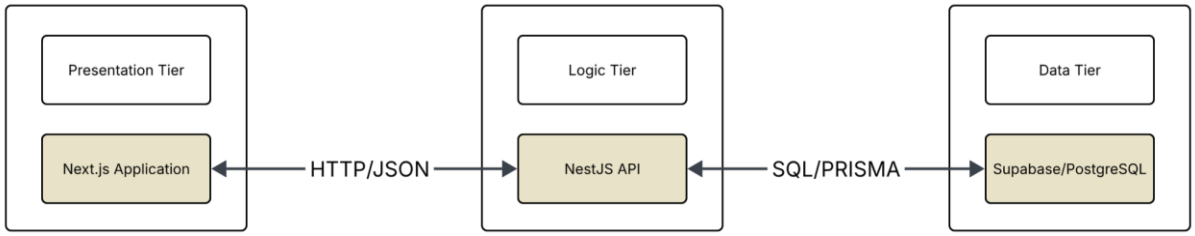

Ilustración 1. Diagrama de arquitectura Multitier. Representación de las tres capas que componen la arquitectura del sistema: presentación, lógica de negocio y datos.

2.3 Especificación de Requisitos de Software (ERS)

2.3.1 Perspectiva del Producto

El **Módulo de Gestión y Matching de Comisiones** es una solución diseñada para optimizar la organización académica en entornos digitales. Puede operar de forma autónoma o integrarse a sistemas existentes.

Su función principal es actuar como intermediario entre el registro de estudiantes y la planificación de la oferta docente. Aborda la problemática de la asignación de cupos centralizando el proceso mediante una interfaz web/móvil.

Este sistema no solo gestiona la comunicación, sino que también maneja la documentación legal de los cambios y organiza la programación temporal (calendario) de las comisiones.

2.3.2 Funciones del Producto

Las funciones principales que el software llevará a cabo son:

Gestión de alumnos:

- El profesor puede incorporar, consultar y dar de baja alumnos dentro de su comisión.

Asignación de horarios y eventos al calendario

- El profesor carga horarios de clase, parciales y entregas de TPs. Los alumnos de la comisión los visualizan automáticamente en su calendario personal.

Mensajería interna

- Canal de comunicación directa dentro de la plataforma entre dos alumnos o entre un alumno y su profesor.

Visualización de perfil

- Visualización de Agenda: Implementación de un calendario personal donde el usuario visualizar sus horarios y eventos académicos.
- Carga de materias con correspondiente comisión: Permite cargar las distintas comisiones del usuario correspondiente a cada materia.

Centro de notificaciones:

- Mediante correo electrónico, el usuario será informado sobre nuevas propuestas de intercambio como confirmaciones de match.

Generación de Comprobante:

- Se genera automáticamente un comprobante en formato PDF que contendrá los datos de ambos estudiantes y de las comisiones intercambiadas para garantizar la trazabilidad del proceso.

2.3.3 Características de los Usuarios

Se identifican dos perfiles de usuario principales:

- **Profesor:** Usuario con permisos de edición sobre la comisión, responsable de cargar contenidos, fechas críticas y moderar la lista de alumnos. Posee un nivel de competencia digital medio/alto.
- **Alumno:** Usuario final que busca gestionar su trayectoria académica, realizar intercambios de comisiones y consultar su cronograma. Requiere una interfaz intuitiva y acceso rápido a notificaciones.

2.3.4 Restricciones

- **Seguridad de Datos:** El sistema debe cumplir con normativas de protección de datos personales de los estudiantes.
- **Formato de Salida:** Los comprobantes de matching deben generarse estrictamente en formato PDF para evitar alteraciones.
- **Conectividad:** Las notificaciones por correo electrónico dependen de la disponibilidad de un servidor SMTP externo.
- **Compatibilidad:** La visualización de la agenda debe ser adaptable a dispositivos móviles (Responsive Design).

2.3.5 Requerimientos del Sistema

2.3.5.1 Requisitos Funcionales (RF)

RF-01 – Gestión de autenticación
El sistema debe permitir que los usuarios se autentiquen mediante Supabase Auth y acceder a funcionalidades protegidas según su sesión activa.
RF-02 – Consulta de datos de estudiante
El sistema debe permitir consultar los datos de un estudiante por id_usuario.
RF-03 – Consulta de comisiones de estudiante
El sistema debe permitir consultar las comisiones en las que está inscrito un estudiante.
RF-04 – Consulta de comisiones de profesor
El sistema debe permitir consultar las comisiones asignadas a un profesor.
RF-05 – Consulta de comisión de profesor
El sistema debe permitir consultar una comisión específica asignada a un profesor.
RF-06 – Consulta de detalle de comisión

El sistema debe permitir visualizar el detalle de una comisión (materia, profesor, cupo, horarios, etc.).

RF-07 – Alta de estudiante en comisión

El sistema debe permitir incorporar un estudiante a una comisión, validando existencia y evitando duplicados.

RF-08 – Baja de estudiante de comisión

El sistema debe permitir dar de baja un estudiante de una comisión existente.

RF-09 – Alta de horario recurrente

El sistema debe permitir registrar horarios recurrentes asociados a una comisión.

RF-10 – Baja de horario

El sistema debe permitir eliminar horarios de una comisión.

RF-11 – Alta de evento académico

El sistema debe permitir crear eventos académicos (clase, parcial, entrega) asociados a comisión/materia.

RF-12 – Baja de evento académico

El sistema debe permitir eliminar eventos académicos.

RF-13 – Visualización de conversaciones

El sistema debe permitir listar las conversaciones de un usuario.

RF-14 – Visualización de mensajes

El sistema debe permitir consultar los mensajes de una conversación.

RF-15 – Envío de mensajes

El sistema debe permitir enviar mensajes dentro de una conversación.

RF-16 – Gestión de notificaciones

El sistema debe permitir registrar y consultar notificaciones vinculadas a eventos/intercambios/comprobantes.

RF-17 – Generación de comprobante

El sistema debe permitir generar y almacenar un comprobante PDF de intercambio confirmado.

2.3.5.2 Requisitos No Funcionales (RNF)

RNF-01 – Seguridad de acceso

Las rutas protegidas deben requerir token JWT válido (Bearer) y validar issuer/audience.

RNF-02 – Control de autorización

El sistema debe restringir operaciones según rol (estudiante/profesor/admin) cuando aplique.

RNF-03 – Integridad de datos

La base de datos debe mantener integridad referencial mediante claves foráneas y restricciones de unicidad.

RNF-04 – Validación de entrada

Las entradas de API deben validarse globalmente para evitar datos inválidos o no permitidos.

RNF-05 – Manejo uniforme de errores

La API debe devolver errores en formato consistente (statusCode, message, path, timestamp).

RNF-06 – Documentación técnica

La API debe estar documentada con Swagger para facilitar consumo y pruebas.

RNF-07 – Mantenibilidad

El backend debe conservar estructura modular por dominio para facilitar evolución y cambios.
RNF-08 – Escalabilidad inicial
La arquitectura debe soportar crecimiento incremental de módulos sin rediseño total.
RNF-09 – Rendimiento mínimo
Las operaciones de consulta frecuentes deben responder en tiempos aceptables para uso interactivo académico.
RNF-10 – Disponibilidad en entorno de desarrollo/prueba
El sistema debe permitir la ejecución estable de frontend y backend para validación funcional.
RNF-11 – Trazabilidad
Las operaciones relevantes (como mensajes) deben quedar persistidas y auditables.
RNF-12 – Portabilidad de despliegue
El sistema debe poder configurarse por variables de entorno sin cambios de código.
RNF-13 – Compatibilidad API-Frontend
Los contratos de rutas y payloads deben mantenerse alineados entre frontend y backend.
RNF-14 – Calidad de código
El proyecto debe mantener análisis estático/linting para reducir errores y mejorar consistencia.
RNF-15 – Usabilidad básica
La interfaz debe permitir a los usuarios completar las tareas principales (visualizar, gestionar comisión, mensajería) con flujo claro.

2.4 Plan de Riesgos

Riesgo (ID)	Descripción	Causas	Impacto	Mitigación
R01: Cambio de Requerimientos por Parte de la Cátedra	La cátedra puede solicitar modificaciones en los requisitos, alcance o entregables durante el desarrollo del proyecto.	- Evolución de la comprensión del problema - Cambios en la coordinación académica	- Replanificación del cronograma - Rediseño de funcionalidades ya implementadas - Atrasos en entregables	1. Reuniones semanales con tutor para alinear expectativas 2. Documentación versionada con Git (cada versión etiquetada)

R02: Tiempo Limitado por Otras Materias/Exámenes	Los miembros del equipo tienen otras obligaciones académicas (parciales, trabajos prácticos, finales).	- Sobrecarga académica del cuatrimestre - Concentración de parciales en fechas críticas - Trabajos prácticos de otras materias	- Atrasos en tareas asignadas - Calidad comprometida por prisa - Burnout del equipo	1. MVP funcional temprano 2. Priorizar funcionalidades 3. Tareas bien atomizadas (máximo 4 horas cada una)
R03: Falta de Experiencia con NestJS/Prisma	El equipo no tiene experiencia previa con las tecnologías seleccionadas (NestJS, Prisma, Supabase).	- Tecnologías modernas no vistas en la currícula - Primera experiencia con arquitectura Clean/Hexagonal - Curva de aprendizaje de TypeScript avanzado	- Desarrollo más lento al principio - Posibles errores de arquitectura - Frustración del equipo	1. **proceso de investigación** dedicado en Semana 1: - Tutorial oficial de NestJS (3 horas) - Tutorial de Prisma con PostgreSQL (2 horas) - Ejemplo de Clean Architecture en NestJS (2 horas) 2. Documentación oficial siempre a mano 3. Proyecto ejemplo como referencia

R04: Problemas de Deployment	El equipo puede encontrar desafíos técnicos o de configuración durante el proceso de *deploy* (por ejemplo, dependencias, incompatibilidad de entornos, falta de automatización) , lo que retrasa la puesta en producción.	- Falta de automatización en el proceso de *deploy* - Incompatibilidad entre el entorno de desarrollo local y el de producción - Dependencias o librerías desactualizadas o conflictivas - Configuración manual propensa a errores	- Retraso en la entrega de la funcionalidad completa y demostrable - Demostraciones inestables o fallidas debido al entorno local - Dificultad para integrar el trabajo de diferentes miembros del equipo - Necesidad de dedicar tiempo extra a la solución de problemas de configuración en lugar de desarrollar - Puntuación reducida en la evaluación por problemas de *deploy*	1. Uso de Docker (o similar) para garantizar un entorno consistente desde el inicio. 2. Definición clara y documentación del proceso de *deploy* y requisitos de infraestructura. 3. Uso de variables de entorno (configuración 100% portable) para manejar credenciales y configuraciones específicas. 4. Implementación de CI/CD básico para automatizar y probar el *deploy* tempranamente. 5. Comunicación proactiva con IT/Administración sobre requisitos de red/servidores.

R05: Conflicto de Horarios en Grupo de Trabajo	Los miembros del equipo tienen horarios incompatibles para reuniones sincrónicas.	- Estudiantes con diferentes horarios de cursada - Compromisos personales	- Dificultad para coordinar - Retrasos en toma de decisiones - Desalineación del equipo	1. Documentación *async* detallada en ClickUp 2. *Daily standup* por escrito (Discord) en vez de llamada 3. Flexibilidad en horarios de trabajo 4. *Overlap* mínimo de 2 horas semanales para *pair programming*

Tabla 2. Plan de Riesgos. Identificación y análisis de los principales riesgos del proyecto, incluyendo su probabilidad de ocurrencia, impacto y estrategias de mitigación propuestas.

2.5 Cronograma

2.5.2 Sprint 1: Cimentación y Análisis (Semanas 1-2)

Objetivo: Establecer el marco de trabajo, identificar riesgos y definir comportamiento del sistema.

- **Diana (Dev/Analista):**
  - Definir formalmente el ciclo de vida scrum adaptado al equipo de dos.
  - Realizar planificación detallada en Clickup asignando tiempos a cada tarea
  - Diseñar el diagrama de casos de uso de las funcionalidades en Lucidspark
  - Crear la estructura base del frontend.
- **Nico (Tester/Dev):**
  - Elaborar el plan de riesgos completo: identificación, clasificación, impacto, etc.
  - Realizar la síntesis de herramientas justificando su uso.
  - Configurar el proyecto en Supabase y definir las tablas físicas.

2.5.3 Sprint 2: Diseño detallado y funcionalidad básica (Semanas 3-4)

Objetivo: Finalizar la documentación técnica de diseño y entregar el prototipo funcional de registro

- **Diana:**
  - Diseñar los 3 diagramas de secuencia que muestre la interacción entre objetos para las funciones críticas.
  - diseñar interfaz de usuario para la funcionalidad básica
  - presentar la bibliografía consultada

- **Nico:**
    - Escribir los 2 contratos de operaciones críticas
    - desarrollar el backend para la funcionalidad básica
    - Redactar 3 conversaciones técnicas que representen los flujos principales de las funcionalidades.
- **Ambos**
    - realizar la evaluación de arquitecturas, comparando opciones y justificando la elección

2.5.4 Sprint 3: Desarrollo Principal - Parte A (Semanas 5-6)

Objetivo: Implementar la visualización de la agenda y la lógica de comisiones.

- **Diana:**
    - Desarrollar la visualización de agenda y calendario.
    - Implementar la interfaz de la visualización de comisiones disponibles.
- **Nico:**
    - Desarrollar la lógica de base de datos para filtrar comisiones por usuario (simulado).
    - Crear los endpoints en para alimentar el calendario y el listado de comisiones.

2.5.5 Sprint 4: Desarrollo Principal - Parte B (Semanas 7-8)

Objetivo: Implementar solicitudes, notificaciones y comprobantes.

- **Diana:**
    - Desarrollar la interfaz para la solicitud de cambio de comisión.
    - Crear la vista de notificaciones internas.
- **Nico:**
    - Implementar la lógica de creación de solicitudes y el envío de notificaciones por correo.
    - Desarrollar el servicio de generación de Comprobante PDF .

2.5.6 Sprint 5: Pruebas, Trazabilidad y Cierre (Semanas 9-10)

Objetivo: Validar el sistema, asegurar la trazabilidad y generar la documentación final.

- **Diana:**
    - Realizar la verificación de trazabilidad entre el código frontal y los diagramas de diseño previos.
    - Redactar el Manual de Usuario con capturas de pantalla de las funcionalidades terminadas.
- **Nico:**
    - Realizar el Mapeo al Modelo de Base de Datos Física final para la documentación.
    - Diseñar y ejecutar el Plan de Pruebas (generación y ejecución de casos de prueba).
    - Elaborar la Documentación Técnica de configuración y despliegue.

2.6 Organización de Sprints en ClickUp

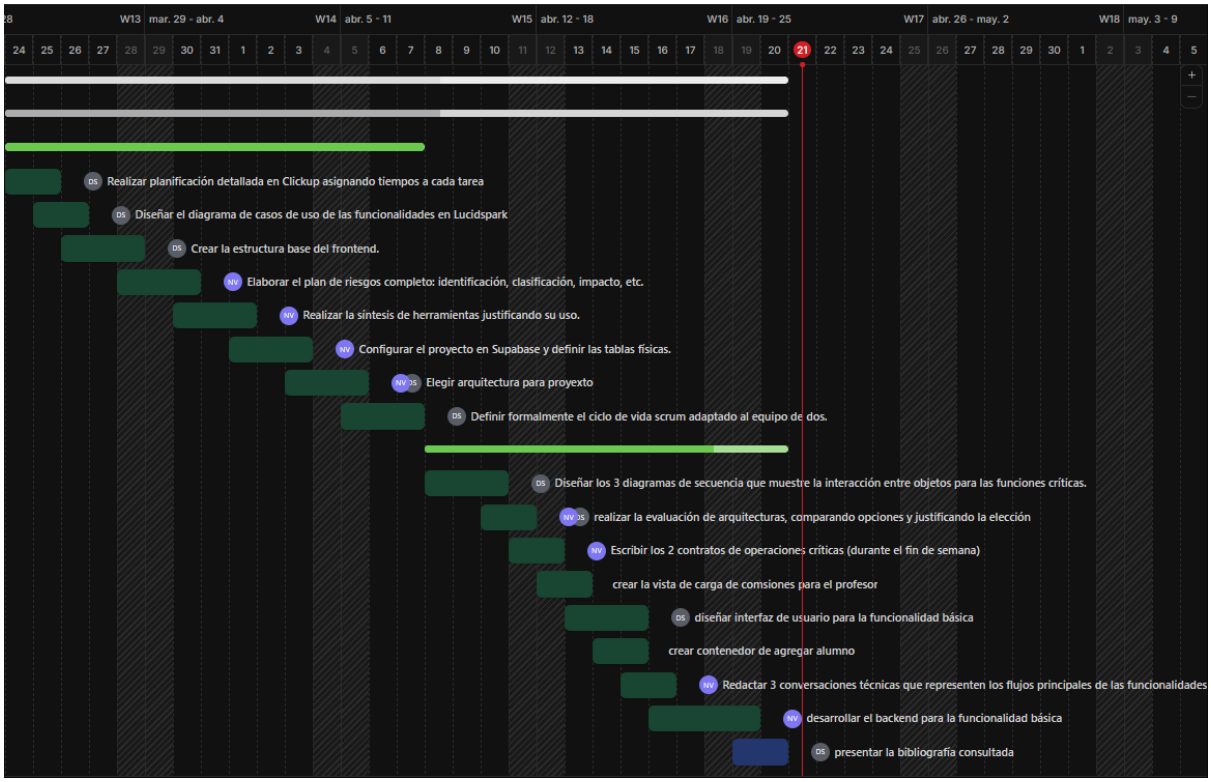

Ilustración 2. Organización de sprints en ClickUp. Captura de la herramienta de gestión que muestra la planificación y seguimiento de los sprints del proyecto.

2.7 Modelo de datos

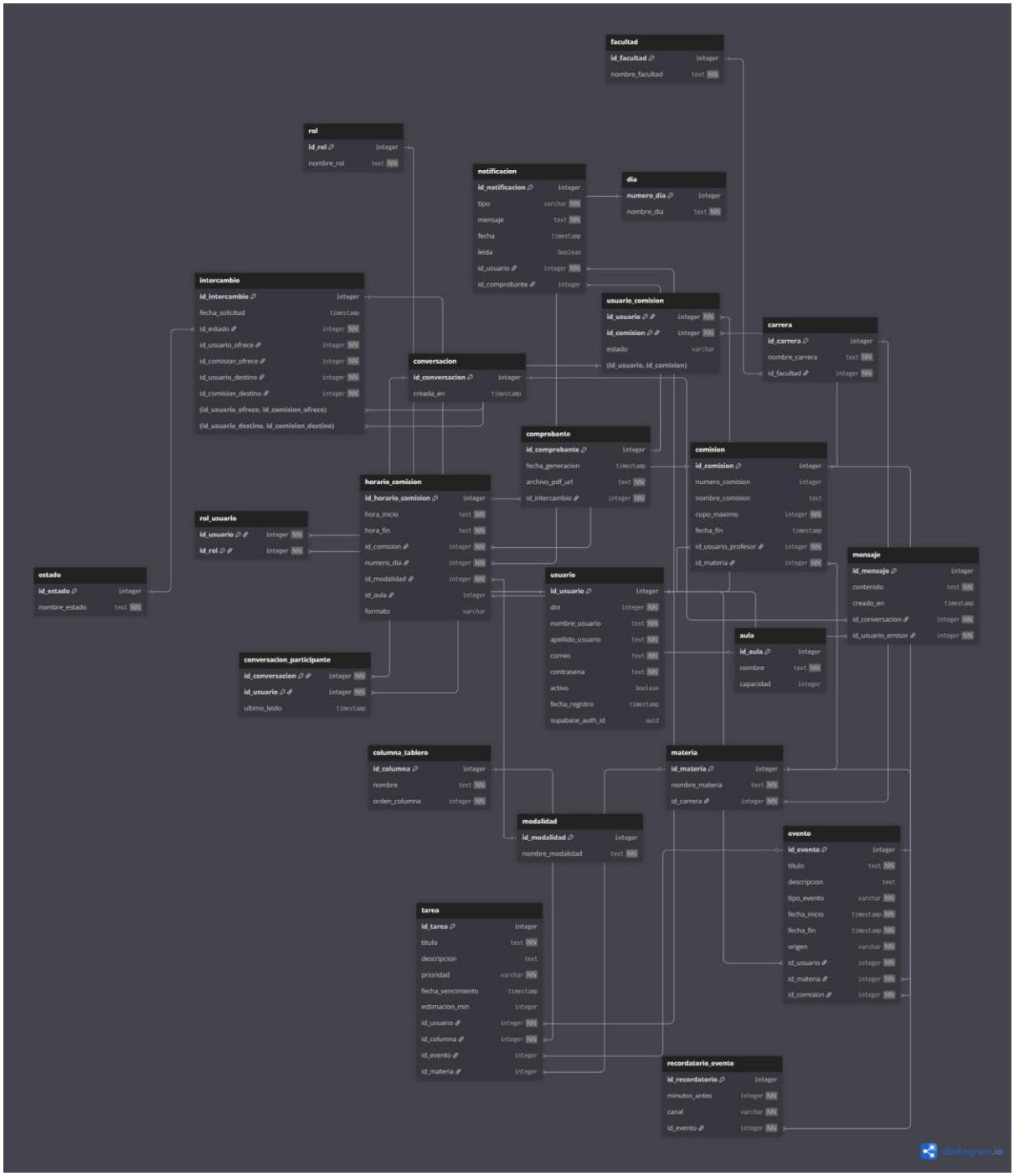

Ilustración 3. Modelo de datos. Diagrama entidad-relación que representa la estructura de la base de datos del sistema de intercambio de comisiones.

3. Herramientas utilizadas

3.1 Stack Tecnológico

3.1.1 Frontend: Next.js 15

Para la interfaz de usuario, la elección de **Next.js 15** asegura un rendimiento de primer nivel. Al contar con enrutado automático, acelera muchísimo los tiempos de desarrollo. Su mayor fortaleza radica en la flexibilidad de renderizado (SSR/CSR), lo que significa que las páginas pueden generarse en el servidor antes de llegar al usuario, logrando tiempos de carga ultrarrápidos y un buen manejo de SEO.

3.1.2 Backend: NestJS + TypeScript

El motor de la aplicación estará construido con **NestJS y TypeScript**. NestJS impone una arquitectura fuertemente modular e inyección de dependencias, lo que hace que el código sea muy ordenado y fácil de mantener a medida que el equipo o el proyecto crecen (es una solución *enterprise-ready*). Al sumar TypeScript, garantizamos un tipado estricto que previene errores críticos antes de que el código llegue a producción.

3.1.3 Base de Datos: PostgreSQL (Supabase)

Los datos vivirán en **PostgreSQL**, gestionados a través del ecosistema de **Supabase**. Esto no solo nos da la robustez de una base de datos relacional tradicional, sino que le suma superpoderes: capacidades *Realtime* para reflejar cambios en la pantalla de los usuarios al instante (sin necesidad de recargar la página) y un módulo de *Storage* integrado que resultará perfecto para alojar y servir los archivos PDF generados por la plataforma.

3.1.4 ORM: Prisma

Para comunicar el Backend con la Base de Datos usaremos **Prisma**. Esta herramienta mejora drásticamente la experiencia de desarrollo (DX). En lugar de escribir consultas SQL manuales propensas a errores, Prisma ofrece un tipado estricto (Type-safety) que autocompleta el código y detecta fallos en tiempo de escritura. Además, sus migraciones automáticas hacen que actualizar la estructura de la base de datos sea un proceso seguro y sin fricciones.

3.1.5 Generación de PDF: Puppeteer

La creación de documentos estará a cargo de **Puppeteer**. Dado que básicamente ejecuta una versión "invisible" del navegador Chrome en el servidor, nos garantiza la máxima calidad de renderizado posible. Cualquier diseño web complejo con CSS avanzado se traducirá exactamente igual al formato PDF, lo que facilita enormemente el mantenimiento: si se ve bien en la web, se verá perfecto en el documento final.

3.1.6 Notificaciones: Supabase Realtime + Nodemailer

El sistema de avisos tendrá un enfoque dual para cubrir todos los frentes. Por un lado, **Supabase Realtime** se encargará de las notificaciones "in-app" (dentro de la aplicación), mensajería, sincronizando alertas en vivo mientras el usuario navega. Por otro lado, **Nodemailer** gestionará el envío de correos electrónicos transaccionales, asegurando que los avisos críticos (como confirmaciones, reseteo de contraseñas o reportes) lleguen directo a la bandeja de entrada de forma confiable.

3.1.7 Testing (Pruebas de Calidad): Jest + Playwright

Para garantizar que el sistema no falle, implementaremos una cobertura de pruebas integral. **Jest** se utilizará para las pruebas unitarias y de integración, asegurando que cada función y módulo del código haga exactamente lo que debe. Para complementar, **Playwright** se encargará de las pruebas *End-to-End* (E2E), simulando el comportamiento de un usuario real haciendo clics y navegando por la app, para verificar que todos los flujos completos funcionen a la perfección.

3.1.8 Animaciones y Diseño: Framer Motion, Stitch y UX-UI-pro

La interfaz no solo debe ser funcional, sino sentirse moderna y profesional. **Framer Motion** aportará animaciones fluidas y declarativas a los componentes de React, dándole vida a la interacción del usuario sin penalizar el rendimiento. Combinado con herramientas como Stitch y metodologías de UX-UI-pro, el resultado será un diseño pulido, intuitivo y con transiciones que elevan la percepción de calidad del producto.

3.1.9 Gestión de Proyecto: ClickUp

La organización del trabajo se centralizará en **ClickUp**. Esta plataforma nos permitirá gestionar los *sprints* de desarrollo de manera ágil, asignar responsables y llevar un tracking detallado del avance de cada tarea. Es fundamental para tener una visión clara de los tiempos de entrega y los cuellos de botella.

3.1.10 Comunicación del Equipo: Discord

Para la interacción diaria utilizaremos **Discord**. Sus canales de voz y texto permiten una coordinación asíncrona rápida, ideal para resolver dudas técnicas en el momento. Además, facilita realizar las reuniones diarias (*daily standups*) de forma dinámica y sin la fricción de programar videollamadas formales.

3.1.11 Diseño y Arquitectura: LucidSpark

Antes y durante el código, la planificación visual vivirá en **LucidSpark**. Aquí construiremos los modelos de casos de uso, los diagramas de secuencia y los esquemas de arquitectura. Es el espacio colaborativo donde el equipo aterrizará la lógica del negocio visualmente antes de programarla, asegurando que todos estemos alineados en cómo funcionará el sistema.

4. Resultados

4.1 Diagrama de Casos de uso

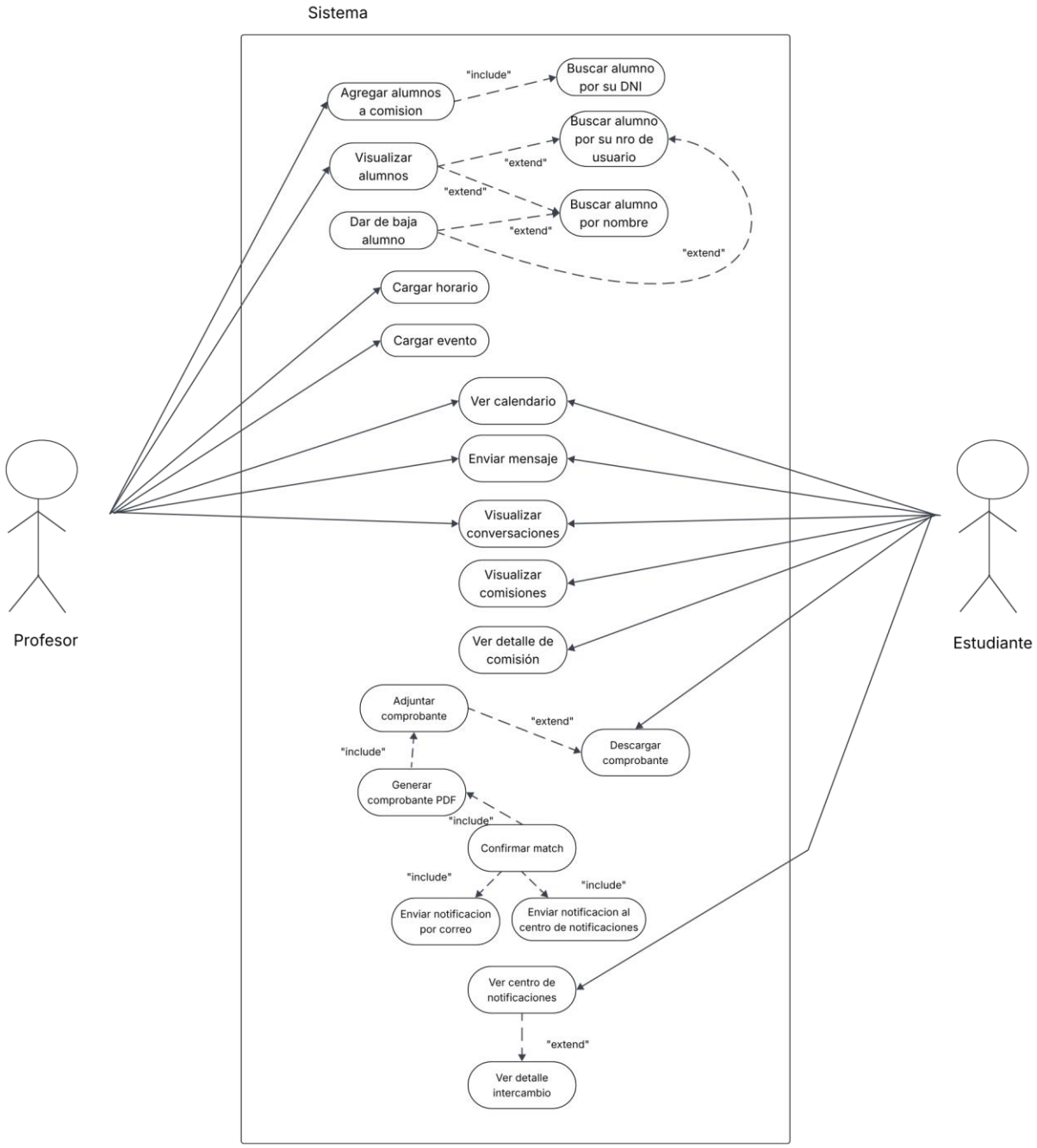

Ilustración 4. Diagrama de casos de uso. Representación unificada de las funcionalidades del sistema y su interacción con los actores Profesor y Estudiante.

4.2 Conversaciones

4.2.1 Conversación 1: Incorporar alumno a comisión

Actor: Profesor

Acción	Curso Normal	Curso Alternativo
1. P: Selecciona la comisión que en donde desea incorporar el alumno.		
2. S: Busca comisión por su número de id.	2.1 S: Encuentra comisión por su número de id. 2.2 S: Muestra el detalle de la comisión.	2.1.1. S: No encuentra comisión por su número de id. 2.1.2 S: Da mensaje de error “No existe comisión con este id” 2.1.3 Volver a 1.
3. P: Ingresa el número de DNI del alumno y selecciona “Buscar”.		
4. S: Busca alumno por número de DNI	4.1. S: encuentra alumno 4.2. S: Muestra nombre, apellido y correo	4.1.1 Alumno no encontrado 4.1.2 S: muestra mensaje "No existe ningún usuario con ese DNI " 2.1.3 Volver a 1
5. P: Selecciona botón "Agregar alumno"		
6. S: Registra alumno en la comisión.	6.1 S: muestra mensaje de éxito de incorporación con nombre, apellido y nro de comisión: “El alumno(nombre y apellido) fue incorporado a la comisión (n°)”	6.1.1 Error al registrar 6.1.2 S: muestra mensaje "Error al incorporar alumno"
7. Fin Caso de Uso		

Tabla 3. Conversación 1: Incorporar alumno a comisión. Descripción del flujo de interacción entre el actor Profesor y el sistema para registrar un alumno en una comisión.

4.2.2 Conversación 2: dar de baja un alumno

Actor: Alumno

Acción	Curso Normal	Curso Alternativo
1. P: Selecciona la comisión que en donde desea incorporar el alumno.		
2. S: Busca comisión por su número de id.	2.1 S: Encuentra comisión por su número de id. 2.2 S: Muestra el detalle de la comisión.	2.1.1. S: No encuentra comisión por su número de id. 2.1.2 S: Da mensaje de error “No existe comisión con este id” 2.1.3 Volver a 1.
3. P: ve la lista “Alumnos inscriptos”		
4. P: Clickea "Dar de baja" en la fila del alumno.		
5. S: Muestra banner de confirmación: "¿Dar de baja a [Nombre]?" con botones Confirmar / Cancelar.	5.1 P: Confirma 5.2 S: da de baja estudiante actualizando el estado de alumno de ACTIVO a BAJA	5.1.1 P Hace clic en Cancelar en el banner ámbar de confirmación. 5.1.2 S: Oculta el banner. El alumno permanece activo. No se

	5.3 S: Muestra banner verde de éxito 5.4 El alumno aparece en sección "Dados de baja" con botón Reincorporar	realiza ninguna llamada al backend. 5.1.3 Volver a 3
6. Fin Caso de Uso		

Tabla 4. Conversación 2: Dar de baja un alumno. Descripción del flujo de interacción entre el actor Profesor y el sistema para eliminar un alumno de una comisión.

4.2.3 Conversación 3: Asignar horario a comisión

Actor: Profesor

Acción	Curso Normal	Curso Alternativo
1. P: Selecciona la comisión que en donde desea asignar un horario.		
2. S: Busca comisión por su número de id.	2.1 S: Encuentra comisión por su número de id. 2.2 S: Muestra el detalle de la comisión.	2.1.1. S: No encuentra comisión por su número de id. 2.1.2 S: Da mensaje de error “No existe comisión con este id” 2.1.3 Volver a 1.
3. P: Selecciona "Calendario"		
4. S: Muestra menú de operaciones en el calendario		
5. p: Completa campos (día, hora de inicio, hora de fin, aula(opcional)) y selecciona "Guardar horario"		
6. S: Registra el horario.	6.1 S:Ubica horario en el calendario 6.2 S: muestra mensaje de éxito con día, hora de inicio, hora de fin, hasta que mes.	6.1.1 S: Encuentra campos incompletos o inválidos 6.1.2 S: muestra mensaje "Complete todos los campos correctamente" 6.2.1 S: Encuentra hora inicio menor a hora fin 6.2.2 S: Muestra mensaje “La hora de fin debe ser mayor a la hora de inicio” 6.1.3 Volver a 5
7. Fin Caso de Uso		

Tabla 5. Conversación 3: Asignar horario a comisión. Descripción del flujo de interacción entre el actor Profesor y el sistema para registrar un horario en el calendario.

4.3 Diagramas de Secuencia

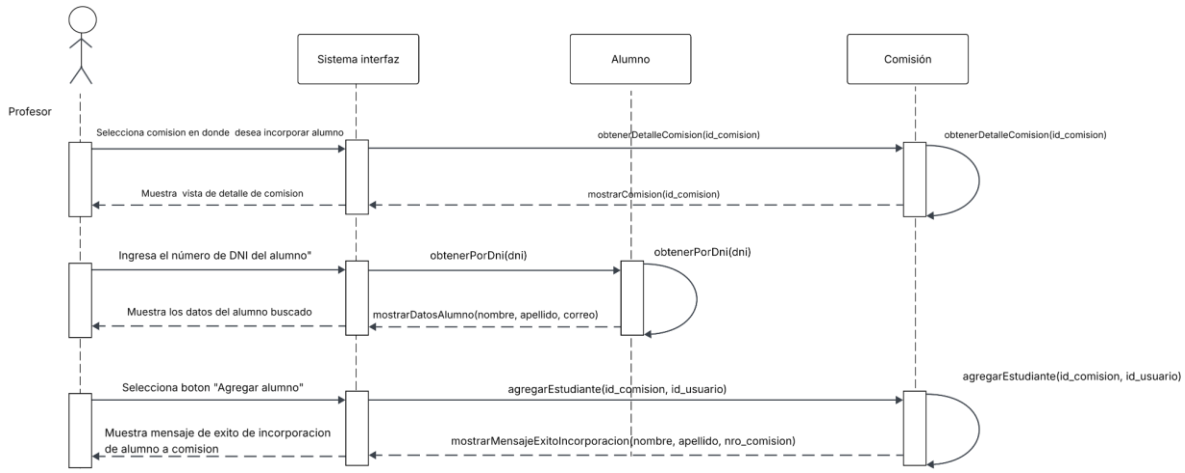

Ilustración 4.1. Diagrama de secuencia: Incorporar alumno a comisión. Secuencia de mensajes entre objetos para registrar un alumno en una comisión, derivada del caso de uso de gestión de alumnos.

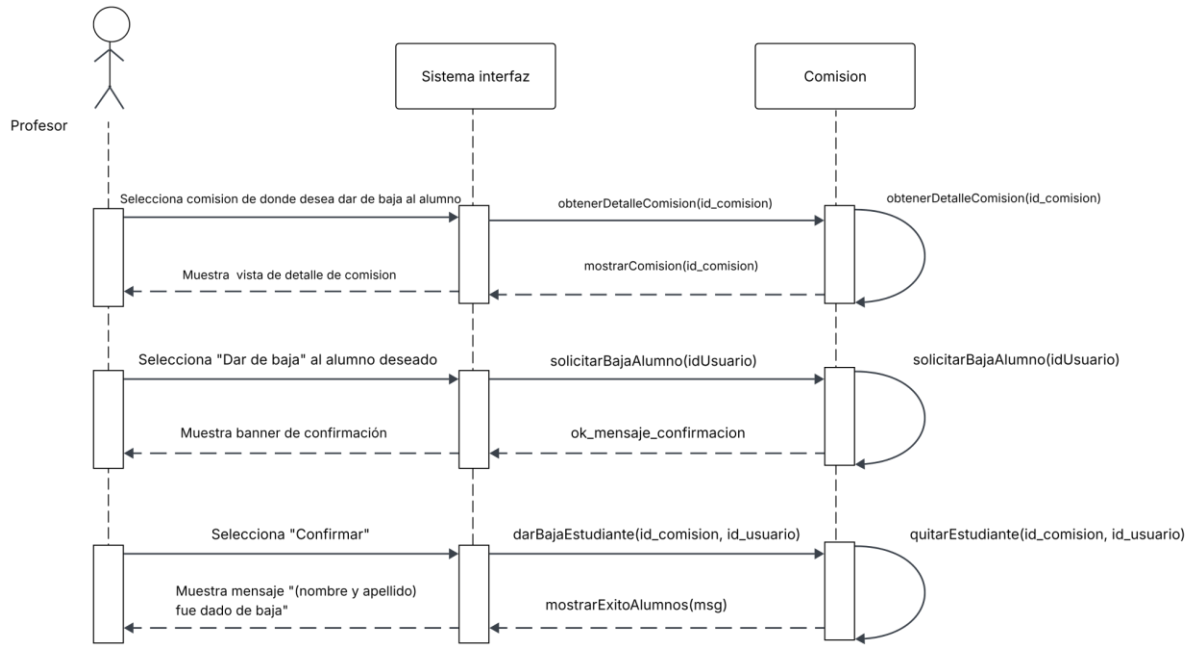

Ilustración 4.2. Diagrama de secuencia: Dar de baja a alumno de comisión. Secuencia de mensajes entre objetos para eliminar un alumno de una comisión, derivada del caso de uso de gestión de alumnos.

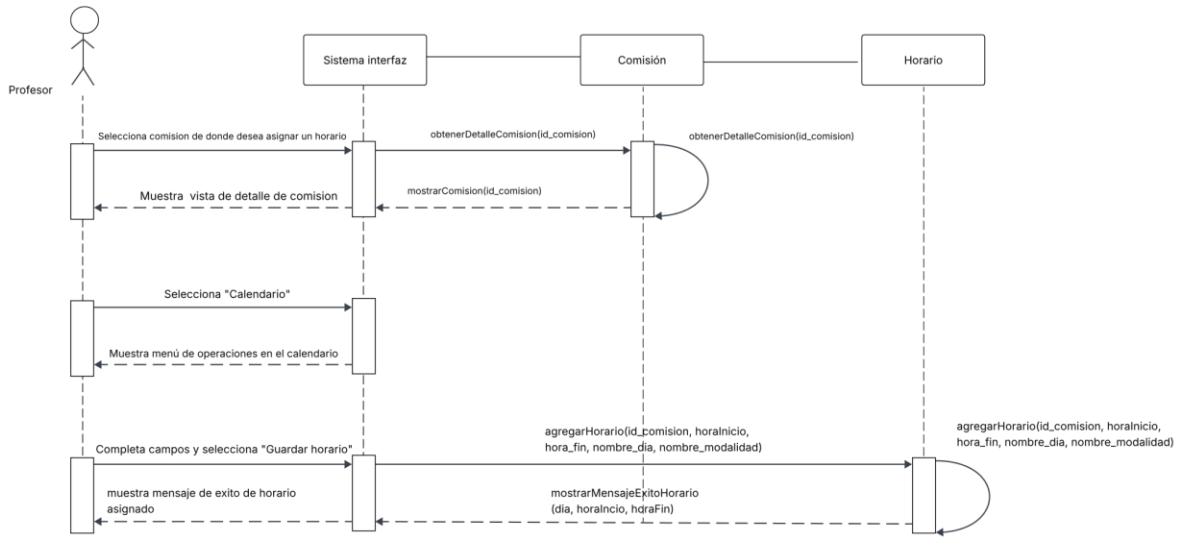

Ilustración 4.3. Diagrama de secuencia: Asignar horario a comisión. Secuencia de mensajes entre objetos para registrar un horario en el calendario, derivada del caso de uso de visualización de calendario.

4.4 Contrato de Operaciones

4.4.1 CO-01: Gestión de Alumnos — Incorporar Alumno

Nombre	agregarAlumno(id_profesor, id_alumno, id_comision)
Referencia Cruzada	Caso de Uso: "Gestión de alumnos"
Responsabilidades	Registrar al alumno como miembro de la comisión del profesor. Verificar que el alumno no esté ya inscripto en esa comisión. Persistir la relación alumno-comisión en el sistema.
Excepciones	Si la sesión es inválida, mostrar error "No autorizado". Si el alumno no existe en el sistema, mostrar error "Alumno no encontrado". Si el alumno ya pertenece a la comisión, mostrar error "Alumno ya registrado". Si la comisión no existe o está inactiva, mostrar error "Comisión no encontrada".
Pre-condiciones	El profesor debe poseer una sesión activa y válida. La comisión debe existir y estar activa en el sistema. El alumno no debe estar ya registrado en esa comisión.
Post-condiciones	Se creó y persistió la relación entre el alumno y la comisión. El alumno puede visualizar la comisión en su perfil.
Invariantes	Un alumno no puede estar registrado más de una vez en la misma comisión. Solo el profesor responsable puede incorporar alumnos a su comisión.

Tabla 6. Contrato de Operación CO-01: Incorporar Alumno. Especificación formal de la operación que permite al profesor registrar un nuevo alumno dentro de su comisión.

4.4.2 CO-02: Asignación de Horarios y Eventos al Calendario

Nombre	asignarEvento(id_profesor, id_comision, tipo, descripcion, fecha, hora)
Referencia Cruzada	Caso de Uso: "Asignación de horarios y eventos al calendario"
Responsabilidades	Registrar el evento o horario en el sistema asociado a la comisión. Publicar automáticamente el evento en el calendario de todos los alumnos de la comisión. Permitir tipos de evento: horario de clase, parcial, entrega de TP.
Excepciones	Si la sesión es inválida, mostrar error "No autorizado". Si la fecha u hora tienen formato incorrecto, mostrar error "Datos inválidos". Si la comisión no existe, mostrar error "Comisión no encontrada".
Pre-condiciones	El profesor debe poseer una sesión activa y válida. La comisión debe existir y tener alumnos registrados. La fecha del evento debe ser igual o posterior a la fecha actual.
Post-condiciones	Se persistió el evento asociado a la comisión. El evento es visible en el calendario personal de todos los alumnos de la comisión.
Invariantes	Solo el profesor responsable puede agregar eventos a su comisión. Todo evento debe tener fecha, hora y tipo definidos.

Tabla 7. Contrato de Operación CO-02: Asignación de Horarios y Eventos. Especificación formal de la operación que permite al profesor cargar horarios y eventos académicos al calendario.

5. Anexos

Las interfaces gráficas, capturas de pantalla de los módulos de gestión docente y del alumnado, se presentan de forma detallada en el documento complementario de anexos para facilitar su consulta y preservar la calidad visual de las capturas.

Se puede acceder al material gráfico a través del siguiente enlace y el drive solicitado**Anexo de Capturas - Sistema de Matching (Módulo 1)**

Drive

Sistema_matching_comisiones

Link de Github: Nicoargento11/matching-comisiones

6. Bibliografía

- Pressman, R. S. (2021). Ingeniería de software (ed. actualizada). McGraw-Hill Interamericana.
- Sommerville, I. (2011). Ingeniería del software (9.ª ed.). Pearson.
- Bass, L., Clements, P., & Kazman, R. (2012). Software architecture in practice (3rd ed.). Addison-Wesley.
- Larman, C. (2003). UML y patrones (2.ª ed.). Alhambra.
- Myers, G. J., Badgett, T., & Sandler, C. (2012). The art of software testing (3rd ed.). Wiley.
- Castaño, M. A. (2000). Diseño de bases de datos relacionales. Omega.

- NestJS. (s. f.). NestJS Documentation. https://docs.nestjs.com/
- Prisma. (s. f.). Prisma Documentation. https://www.prisma.io/docs
- Supabase. (s. f.). Supabase Documentation. https://supabase.com/docs
- Vercel. (s. f.). Next.js Documentation. https://nextjs.org/docs